#### FEEDBACK PAGE

This page retrieves messages from a session and requests comprehensive feedback from the LLM. To improve efficiency and reduce latency, the feedback is cached in Redis under the key `feedback:${sessionId}` to prevent regeneration on subsequent visits.

#### INTERVIEWSIDEBAR

This component lists all interview sessions for a logged-in user. It uses the `fetchInterviewSessions` server action to fetch session data from Redis. It then displays session details like job type, date, difficulty, and question type. The following listing shows the contents of this function.

**Listing 10.1 Function to retrieve all the past interview sessions**

```
export async function fetchInterviewSessions(userId) {
  const sessionIds =
await redis.smembers(`user:sessions:${userId}`);
  const sessions = await Promise.all(
    sessionIds.map(async (sessionId) => {
      const session = await redis.hgetall(`session:${sessionId}`);
      return { ...session, id: sessionId };
    }),
  );
  sessions.sort((a, b) => b.createdAt - a.createdAt);
  return sessions;
}
```

**Retrieves all session IDs associated with a given user from Redis**

**Iterates over each session ID, fetching the session details from Redis in parallel**

**Sorts the sessions in descending order based on the createdAt timestamp. This ensures the most recent sessions are displayed first.**

In the given code, `user:sessions:${userId}` is a Redis key used to store a set of session IDs associated with a specific user. A Redis set is used here due to uniqueness, since we don't want duplicate session IDs for a user.

The `redis.smembers(user:sessions:${userId})` command connects to the Redis server and executes the `SMEMBERS` command. `SMEMBERS` retrieves all members of the set stored at the specified key. The result, `sessionIds`, will be an array of strings, where each string is a session ID. Then, for each `sessionId`, we fetch the session details. The result is then sorted by date in descending order, thus bringing the most recent sessions first in the list.

#### TEXT-TO-SPEECH API ROUTE

The `/api/tts` API route uses the Google Cloud Text-to-Speech API to convert text into natural-sounding audio. This route streams the audio content back to the client for playback. We opt for an API route over a server action to showcase the flexibility of the framework. A feature flag, controlled by the `NEXT_PUBLIC_FEATURE_TTS_ENABLED` environment variable, enables or disables the entire text-to-speech functionality, including the audio controls in the user interface.